ENGINEERING DEPLOYMENT PLAN · REV 1.0

# EchoPath on NVIDIA Jetson Nano

Porting the real-time assistive navigation pipeline from an RTX 4500 Ada workstation to the Tanna TechBiz **Eagle 101** Jetson Nano full system

| TARGET HARDWARE | SOURCE PIPELINE | PHASES | BLOCKERS LOGGED |
|---|---|---|---|
| Eagle 101 + Jetson Nano (eMMC) | YOLOv8n v3 + MiDaS Small | 0 → 8 (nine stages) | 20 (3 critical) |

## Executive Summary

EchoPath currently exists as a set of Jupyter notebooks that batch-process recorded video on a Windows workstation with an NVIDIA RTX 4500 Ada GPU. The published results — 29 FPS, 20.6 ms/frame, mAP@50 = 0.869 — were all measured on that workstation. The deployment target is a fundamentally different class of machine, and the gap between the two is the substance of this document.

The single most important finding is that **the current software stack cannot run on the target hardware at all**. The Eagle 101 is a carrier board for the *original* NVIDIA Jetson Nano module (Tegra X1, Maxwell architecture, 128 CUDA cores, 4 GB shared LPDDR4). That module is end-of-life and permanently capped at JetPack 4.6.x — Ubuntu 18.04, **Python 3.6**, CUDA 10.2, TensorRT 8.2. PyTorch 2.4.1, Ultralytics 8.3, and Transformers 5.x all require Python 3.8 or newer and will not install. There is no version-bumping path out of this; the constraint is architectural.

The consequence is that deployment is **not** an installation exercise. It is a port. The chosen strategy is:

> **Deployment strategy in one sentence**
>
> Export both models to ONNX on the Windows workstation, build TensorRT engines *on the Nano itself*, and execute inference through the TensorRT Python runtime with PyCUDA — with **no PyTorch and no Ultralytics installed on the device** — behind a backend abstraction that lets the identical fusion and alert logic run on both machines.

A second constraint compounds this: the Nano's Maxwell GPU is compute capability **5.3**, which lacks the DP4A instruction TensorRT requires for INT8. **FP16 is the precision floor** — there is no INT8 path on this hardware, so one of the usual edge-optimisation levers is simply unavailable and input resolution has to carry more of the load. This is logged as `B20`.

A third finding, independent of the software stack, is a **compute deficit**. On a Maxwell-generation Nano, YOLOv8n at 640 px in FP16 TensorRT lands at roughly 12–18 FPS *on its own*. Adding MiDaS Small and running the current strictly-serial per-frame loop yields an estimated **4–6 FPS** unoptimized. Phases 2, 4 and 5 exist to recover that, with a realistic target of **10–15 FPS** end-to-end. The 29 FPS figure in Table I of the report is not achievable on this hardware and the paper must be amended to say so.

A fourth finding is that several behaviours the paper describes are **not actually implemented** — most notably the 2.5-second alert cooldown and proximity-based priority queue of Section III-B. The offline notebook approximates them with whole-video string de-duplication, which has no live-operation equivalent. These must be written from scratch before the system can be worn by a person.

### STRUCTURE OF THIS DOCUMENT

- **Section 1** — confirmed hardware profile of the target platform.
- **Section 2** — assessment of the existing codebase, and what survives the port.

- **Section 3** — **Blockers register**: all 19 identified blockers, each with severity, root cause, resolution procedure, and a verification test.
- **Section 4** — the nine-phase execution plan, with tasks, deliverables and exit criteria per phase.
- **Section 5** — degradation ladder and risk register.
- **Section 6** — **the geometric depth alternative**: replacing MiDaS with projective geometry, and how to validate it before the board arrives.
- **Section 7** — required amendments to the report/paper.
- **Appendices** — command reference, configuration sketch, alert vocabulary enumeration, schedule.

---

## 1 · Target Platform — Confirmed Hardware Profile

The following is confirmed from the retail packaging of the delivered unit (Tanna TechBiz Eagle 101 Nano Base Full System, Serial 1010425035310), cross-referenced against the vendor product listing.

| Attribute | Confirmed value | Engineering consequence |
|---|---|---|
| **Module** | NVIDIA Jetson Nano (original), Tegra X1 | Maxwell GPU, 128 CUDA cores, ~472 GFLOPS FP16. Quad-core ARM Cortex-A57 @ 1.43 GHz. EOL — JetPack 4.6.x is terminal. |
| **Memory** | 4 GB LPDDR4, *shared* CPU/GPU | Inferred with high confidence: the eMMC variant is the production module P3448-0020, which is 4 GB only. There is no 2 GB eMMC module. Shared memory means the depth map, detection buffers and framebuffer all compete. |
| **Storage** | **16 GB eMMC** on-module + 1× microSD slot | **Critical constraint.** Full JetPack 4.6 is ~14 GB. See blocker `B2`. |
| **Camera** | 2× MIPI CSI-2, 2-lane, 15-pin | IMX219-compatible. Use `nvarguscamerasrc` — hardware ISP, zero-copy NVMM buffers. Strongly preferred over USB UVC. |
| **USB** | 4× USB 3.0 Type-A; 1× USB Type-B *data only* | Type-B is the **USB recovery / flashing port**. Type-A ports host the audio dongle and any dev peripherals. |
| **Display** | 1× HDMI, 1× DisplayPort | Used only during bring-up. Production runs headless. |
| **Audio** | **None listed** — no analog codec | Audio is available only embedded in HDMI/DP. A USB audio dongle is a **mandatory BOM item**. See blocker `B5`. |
| **Power** | 5 V / 4 A barrel input | 20 W headroom. Budget ~10 W sustained under MAXN with both engines active. |
| **Cooling** | Active fan included (visible on unit) | Good — mandatory for sustained load. Confirm the fan header is driven by the vendor device tree. |
| **Expansion** | M.2 Key E, 40-pin GPIO header, Gigabit Ethernet | Key E is Wi-Fi/BT only — *not* NVMe. GPIO header is the mounting point for the physical mute/repeat button (Phase 8). |

### 1.1 Software ceiling

| Component | Declared in `pyproject.toml` | Jetson Nano ceiling | Verdict |
|---|---|---|---|
| Python | ≥3.11, <3.13 | **3.6** (JetPack 4.6 system) | Incompatible |

| Component | Declared in `pyproject.toml` | Jetson Nano ceiling | Verdict |
|---|---|---|---|
| PyTorch | `torch==2.4.1` (cu118) | **1.10.0** — last cp36 aarch64 wheel | Incompatible |
| Ultralytics | `>=8.3.0` | requires Python ≥3.8 | Will not install |
| Transformers / HF Hub | `>=5.0.0` / `>=1.0.0` | — | Unusable (also unneeded) |
| NumPy | `>=1.26,<2.0` | ≤1.19.x on py3.6 | Incompatible |
| OpenCV | `opencv-python>=4.10` (pip) | 4.1.1 shipped by JetPack, built with GStreamer + CUDA | Do *not* pip-install; it shadows the working build |
| CUDA / cuDNN / TensorRT | 11.8 / — / — | 10.2 / 8.2 / **8.2** | Use TensorRT directly |
| pyttsx3 | `>=2.90` | Installs; backend becomes `espeak`, not SAPI5 | Works, but replace — see `B14` |

**The load-bearing conclusion**

Nothing in the current dependency tree above the CUDA line survives the port. The device-side runtime is: **system Python 3.6 + JetPack's OpenCV + NumPy 1.19 + `tensorrt` (system bindings) + `pycuda`**. That is the entire inference stack. Everything else — training, export, calibration, TTS pre-rendering, evaluation — stays on the Windows workstation.

## 2 · Current Codebase Assessment

### 2.1 Inventory

| Artefact | What it is | Disposition |
|---|---|---|
| `TTS_integrated_model.ipynb` | The real system: YOLO + MiDaS fusion, alert generation, pyttsx3 TTS, 5-pass ffmpeg audio mux. 18 cells. | **Source of truth.** Cells 7 & 13 hold the logic worth keeping; cell 15 is offline-only and gets replaced wholesale. |
| `Yolo_train.py` | Ultralytics training entry point for run v3. | Keep on workstation. Reused for the imgsz fine-tune in Phase 2. |
| `echopath_test.ipynb` | Four near-identical `model.predict()` smoke cells against v1/v2/v3 weights. | Superseded by the Phase 1 CLI. Archive. |
| `runs/detect/runs/echopath/v3/weights/best.pt` | Final deployment weights (6 MB). | **Pin this as the single deployment artefact.** |
| `midas_small/*.ipynb` (7 files) | Pipeline variants — absolute depth, hybrid, TTS, updated, fixed… | Experimental lineage. Freeze; do not port. Extract any threshold learnings into `config.yaml`. |
| `Depth Analysis V2/`, `Depth Anything Indoor/` | The Depth Anything V2 vs MiDaS comparison behind report Figs. 7–8. | Paper evidence. Not deployed — MiDaS Small is the selected model. |
| `Video_Data/` (13 videos) | Academic Block Building test corpus. | **Becomes the Phase 7 fidelity benchmark and the evaluation set for the geometric baseline (§6).** |
| `dataset_merged_v3/.../data.yaml` | Declares `nc: 19`. | Conflicts with the paper's 17. See `B10`. |
| `main.py` | `print("Hello from echopath!")` | Empty stub. Becomes the CLI entry point. |

### 2.2 What the fusion logic actually does

Cell 7 of `TTS_integrated_model.ipynb` is the algorithmic core and is genuinely good work. It must be preserved *semantically unchanged* so the paper's results continue to describe the deployed system:

- `clamp_bbox()` — clips boxes to frame bounds with a degenerate-box guard. Necessary for tall objects (columns, doors) that overrun the frame.
- `get_depth_and_zone()` — takes the **inner 50 % crop** of each box to avoid background wall bleed, then the **median** (not mean) depth for outlier robustness, then classifies against **adaptive per-frame percentile thresholds** (P75 → close, P50 → medium). Direction comes from the box centroid against vertical thirds.
- `generate_alerts()` — filters to `close` / `medium` only, sorts close-first.

Three of these four ideas are exactly right for an embedded port. The fourth — adaptive per-frame percentiles — is the source of two separate problems (`B11` performance, `B12` stability) and is addressed without abandoning the concept.

### 2.3 What does not survive

1. **The whole of cell 15.** It is an offline batch driver: open video file → write annotated frames to a temp MP4 → second pass to synthesise WAV clips → third pass to re-encode silently → fourth pass to build a silent base track → fifth pass to `adelay` / `amix` every clip at its frame timestamp. None of this has a live-camera analogue. A wearable has no timeline to mix against.
2. **Hardcoded absolute Windows paths** ( `E:\EchoPath\…` ) in every notebook.
3. **The annotation path.** `cv2.rectangle` + `cv2.putText` per detection per frame, plus `cv2.VideoWriter`, are pure overhead when the user is blind. Retained behind a debug flag only.
4. **Video rotation machinery** ( `get_video_rotation` , `fix_rotation` , the ffmpeg `displaymatrix` regex). A fixed chest-mounted CSI camera has a fixed, known orientation set once in config.
5. **The `TTSEngine` as written.** It only has `save()` — synthesise-to-file. There is no `speak()` . See `B8` .

### 2.4 Target package layout

```
echopath/
  __init__.py
  config.yaml          # thresholds, class policy, camera params, engine paths
  config.py            # typed loader + validation
  camera.py            # CameraSource ABC -> CsiCamera (GStreamer) | FileSource | UsbCamera
  detector.py          # Detector ABC -> UltralyticsDetector (dev) | TensorRTDetector (Nano)
  depth.py             # DepthEstimator ABC -> MidasTorch (dev)   | MidasTensorRT (Nano)
  fusion.py            # clamp_bbox, get_depth_and_zone, associate  <- lifted from cell 7
  geometric.py         # projective distance estimator + IoU tracker  (§6)
  alerts.py            # AlertPolicy: hysteresis, cooldown, priority, class filter   <- NEW
  audio.py             # WavVocabularyPlayer (primary) | Pyttsx3Fallback
  nms.py               # numpy NMS (TensorRT 8.2 path emits raw head output)
  runtime.py           # threaded stage graph + drop-stale queues
  profiler.py          # per-stage latency ring buffer, tegrastats hook
  cli.py               # echopath run --source csi|file --debug-view --profile
tools/                 # workstation-only
  export_onnx.py       # YOLOv8n + MiDaS -> ONNX, with numerical parity check
  proto_geometric.py   # bbox-geometry distance prototype + approach plots (§6)
  ablate_imgsz.py      # model.val() sweep over {640,512,416,320}
  render_vocabulary.py # pre-synthesise the 102-phrase alert WAV set
  compare_alert_logs.py# Phase 7 workstation-vs-Nano fidelity diff
assets/audio/*.wav     # 102 pre-rendered alert phrases
engines/               # device-built .plan files (gitignored — not portable)
```

The load-bearing design decision is the **backend abstraction**. `fusion.py` and `alerts.py` contain zero framework-specific code and run byte-identically on both machines. Only `detector.py` and `depth.py` have two implementations. This is what makes the Phase 7 fidelity comparison meaningful — any behavioural difference is attributable to numerics (FP16 rounding, resolution, frame-skip) rather than to divergent logic.

## 3 · Blockers Register

Twenty blockers are logged. Severity is defined by impact on the deployment, not by effort to fix.

| ID | Blocker | Severity | Phase | Class |
|---|---|---|---|---|
| `B1` | Software stack incompatible with Python 3.6 ceiling | CRITICAL | 0, 2, 3 | Platform |
| `B2` | 16 GB eMMC too small for JetPack 4.6 + swap + engines | CRITICAL | 0, 3 | Platform |
| `B3` | Production eMMC module cannot be flashed from Windows | CRITICAL | 0, 3 | Logistics |
| `B4` | Third-party carrier requires vendor BSP / device tree | HIGH | 0, 3 | Logistics |
| `B5` | No audio output hardware on the board | HIGH | 0 | Hardware |
| `B6` | Compute deficit — ~4–6 FPS vs the paper's 29 FPS | HIGH | 2, 4 | Performance |
| `B7` | 4 GB shared RAM; TensorRT engine build will OOM | HIGH | 0, 3 | Platform |
| `B8` | Alert cooldown & priority described in paper but unimplemented | HIGH | 5 | Correctness |
| `B9` | `chair` recall 0.30 — primary trip hazard largely undetected | HIGH | 6 | Safety |
| `B10` | `data.yaml` declares 19 classes; paper claims 17 | MEDIUM | 1, 6 | Correctness |
| `B11` | Full-resolution depth upsample + per-box percentile recomputation | MEDIUM | 4 | Performance |
| `B12` | Proximity zone flicker from per-frame normalisation | MEDIUM | 5 | Correctness |
| `B13` | MiDaS loaded via `torch.hub` requires internet at startup | MEDIUM | 2 | Correctness |
| `B14` | `pyttsx3` falls back to espeak on ARM — slow, robotic, CPU-competing | MEDIUM | 5 | Performance |
| `B15` | TensorRT engines are not portable across TRT version or GPU arch | MEDIUM | 2, 3 | Process |
| `B16` | Thermal throttling and battery budget for wearable operation | MEDIUM | 0, 7 | Hardware |
| `B17` | System is notebook-bound with hardcoded absolute paths | MEDIUM | 1 | Process |
| `B20` | Maxwell GPU has no INT8 support — FP16 is the precision floor | MEDIUM | 2, 3 | Platform |
| `B18` | Class imbalance ≈1800:1 renders some reported metrics meaningless | LOW | 6 | Data |
| `B19` | No failure recovery, no user controls, no service supervision | LOW | 8 | Robustness |

### B1 Software stack incompatible with the Python 3.6 ceiling CRITICAL

**ROOT CAUSE**

The Jetson Nano module is EOL. Its final JetPack is 4.6.x (L4T 32.7), which ships Ubuntu 18.04 with system Python 3.6, CUDA 10.2 and TensorRT 8.2. Python 3.6 is a hard floor for the whole toolchain: PyTorch dropped cp36 wheels after 1.10.0, and NVIDIA's last aarch64 build for this platform is `torch-1.10.0a0+git36449ea-cp36-cp36m-linux_aarch64.whl`. Ultralytics requires Python ≥3.8 and therefore cannot be installed under any configuration.

**WHY IT BLOCKS**

Every naive deployment path — `pip install -r requirements.txt`, `YOLO(...).predict()`, `torch.hub.load('intel-isl/MiDaS', …)` — fails at the first step. Attempting to build Python 3.8 + PyTorch from source on the device is a multi-day exercise that produces an unsupported, fragile stack and still leaves Ultralytics fighting NumPy 1.19.

**RESOLUTION**

1. Accept that **no deep-learning framework is installed on the device**. Inference runs on the TensorRT C++/Python runtime, which is a first-class JetPack component and needs nothing from PyPI.
2. Do all model work on the Windows workstation: train (already done), export to ONNX, and verify numerically.
3. On the device install only: `pycuda`, and a virtualenv created with `python3 -m venv --system-site-packages` so it inherits JetPack's `tensorrt` and `cv2` bindings. NumPy stays at the system 1.19.x.
4. Implement post-processing (decode + NMS) in NumPy inside `nms.py`, since the TensorRT 8.2 `EfficientNMS` plugin path on JetPack 4.6 is unreliable.

**VERIFICATION**

`python3 -c "import tensorrt, pycuda.autoinit, cv2, numpy; print(tensorrt.__version__, cv2.__version__)"` succeeds inside the venv, and `cv2.getBuildInformation()` reports `GStreamer: YES`.

### B2 16 GB eMMC is too small for JetPack 4.6 plus swap and engines CRITICAL

**ROOT CAUSE**

The delivered unit carries the production module with 16 GB eMMC (≈14.6 GiB usable after formatting). A complete JetPack 4.6 installation consumes approximately:

| Component | Approx. size |
|---|---|
| L4T rootfs (Ubuntu 18.04 base + desktop) | ~6.0 GB |
| CUDA 10.2 toolkit | ~2.5 GB |
| cuDNN 8.2 | ~1.5 GB |
| TensorRT 8.2 | ~1.5 GB |
| OpenCV 4.1.1, VPI, Multimedia API, samples, docs | ~2.5 GB |
| **Total** | **≈14.0 GB** |

**WHY IT BLOCKS**

That leaves under 1 GB free — before the ~6 GB swapfile that `B7` requires, before the TensorRT `.plan` files, before the 13 validation videos needed for Phase 7, and before any build artefacts. The device will run out of space during the first engine build and the failure mode is confusing (silent `trtexec` abort, or an unbootable system if `/` fills).

**RESOLUTION — RECOMMENDED**

**Flash the root filesystem onto the 128 GB microSD card, leaving the bootloader on eMMC/QSPI.** This is a supported Jetson Nano configuration (the `flash.sh` rootfs-device target is set to the SD device rather than

`mmcblk0p1`). The storage problem then disappears entirely: full JetPack, an 8 GB swapfile, all engines, the complete `Video_Data` corpus, and headroom for iteration.

The trade-off is read throughput — a 140 MB/s UHS-I card versus roughly 300 MB/s eMMC. For a load-once/run-forever workload this affects boot time and nothing else. Use an **A2-rated** card; A1 cards have markedly worse random IOPS and will make `apt` operations painful.

#### RESOLUTION — FALLBACK

Keep rootfs on eMMC but flash a *minimal* configuration: omit samples, documentation, VisionWorks and VPI. Then relocate `/home`, the swapfile, `engines/` and `Video_Data/` to the microSD via `/etc/fstab` bind mounts. Workable, but leaves you managing free space for the remainder of the project.

#### VERIFICATION

`df -h /` reports >40 GB available after JetPack install; `free -h` shows the full swap active; a full FP16 `trtexec` build completes without `ENOSPC`.

### B3 The production eMMC module cannot be flashed from Windows CRITICAL

#### ROOT CAUSE

The familiar "download the `.img`, write it with balenaEtcher, insert the card and boot" workflow applies **only to devkit modules** (P3448-0000, microSD-based). The production eMMC module has no such image. It must be flashed over **USB recovery mode** through the Type-B port, driven by NVIDIA SDK Manager or `flash.sh` running on an **x86_64 Linux host**.

#### WHY IT BLOCKS

The team's development machine is Windows. Without an appropriate host machine the board cannot be brought up at all, and this is a *procurement/logistics* dependency with real lead time — which is why it must be started on day one, in parallel with software work that needs no hardware.

#### RESOLUTION

1. Provision an **x86_64 host running Ubuntu 18.04 or 20.04**. These are the versions SDK Manager supports for JetPack 4.6; newer Ubuntu releases have glibc and dependency conflicts.
2. **Use bare metal or a dual boot, not a virtual machine.** During flashing the board re-enumerates on the USB bus several times as it transitions between recovery and bootloader states. VM USB passthrough commonly drops the device across a re-enumeration, and an interrupted flash can leave the module in a state that requires a full recovery cycle to escape. WSL2 is not viable — it has no native USB device access for this.
3. Put the board into recovery mode per the vendor's assembly guide (typically jumper `FC REC` to `GND`, then apply power), connect the Type-B cable, and confirm with `lsusb` — an `NVIDIA Corp. APX` device must appear.
4. Flash using the vendor BSP (see `B4`), targeting the microSD as rootfs device (see `B2`).

#### VERIFICATION

`lsusb | grep -i nvidia` shows the APX device before flashing; after flashing the board boots to a login prompt on HDMI and `cat /etc/nv_tegra_release` reports an R32.7.x release.

### B4 Third-party carrier board requires the vendor BSP and device tree HIGH

#### ROOT CAUSE

NVIDIA's stock L4T package contains a device tree for NVIDIA's own reference carrier. The Eagle 101 differs in pin multiplexing, CSI lane routing, USB hub topology, fan PWM control and power sequencing. Tanna TechBiz publishes a board support package for exactly this reason — the packaging carries a dedicated "*How to Flash JetPack (BSP)*" QR code.

#### WHY IT BLOCKS

Flashing stock L4T onto a third-party carrier typically *appears* to work — the board boots to a console — and then silently fails at everything that matters: no CSI cameras enumerate, the fan never spins (leading straight to thermal throttling under load), and some USB ports are dead. These symptoms are easy to misdiagnose as camera or cable faults and can burn days.

RESOLUTION

1. Scan the **"How to Flash JetPack (BSP)"** QR code on the retail box and follow the vendor procedure verbatim.
2. Also scan **"How to Assemble"** before seating the module — the SO-DIMM latch and fan header orientation are easy to get wrong once.
3. If the online documentation is thin, email `sales@tannatechbiz.com` and request: the BSP archive, the exact `flash.sh` board-configuration name, and confirmation of the supported rootfs-on-SD invocation.
4. **Do not copy a generic `flash.sh jetson-nano-emmc mmcblk1p1` command from a forum post.** The board-config name is vendor-specific, and an incorrect config on a third-party carrier is the one step in this entire plan that can leave the unit non-booting.

VERIFICATION

After flashing: the fan spins up under `jetson_clocks`; `ls /dev/video*` shows the CSI device; a `nvarguscamerasrc` GStreamer pipeline delivers frames; all four USB 3.0 ports enumerate a test device.

### B5 No audio output hardware on the board HIGH

ROOT CAUSE

The Jetson Nano module carries no analog audio codec, and the Eagle 101 exposes audio only as an HDMI/DisplayPort embedded stream. The packaging lists no audio jack. EchoPath's entire output modality is spoken audio delivered to earphones.

WHY IT BLOCKS

Without an audio sink the system produces no user-perceptible output whatsoever. This is a missing BOM item, not a software problem, and it is absent from the report's hardware description.

RESOLUTION

- **Primary:** a USB audio dongle (any generic C-Media/USB Audio Class 1.0 device). Plug-and-play under L4T, costs very little, and four USB 3.0 ports are available. Set it as the default ALSA sink in `/etc/asound.conf` so the application need not care.
- **Alternative:** an I2S codec (e.g. a ReSpeaker-class HAT) on the 40-pin header — better latency and it frees a USB port, but requires device-tree work on top of the vendor BSP. Not worth it for this project.
- **Avoid Bluetooth audio** for the wearable: A2DP adds 100–200 ms of buffering latency directly onto the alert path, which undermines the report's sub-one-second end-to-end claim, and pairing reliability during movement is a support burden.
- Pin the sink by USB path rather than card index — index ordering is not stable across reboots.

VERIFICATION

`aplay -l` lists the USB device; `aplay -D default assets/audio/person_close_ahead.wav` is audible in the earphones; the device still resolves correctly after three consecutive reboots.

### B6 Compute deficit — realistic throughput is far below the published 29 FPS HIGH

ROOT CAUSE

Report Table I reports 29 FPS and 20.6 ms/frame. Section III-E makes clear these were measured on an **NVIDIA RTX 4500 Ada (24.6 GB)**. The Jetson Nano's Maxwell GPU delivers roughly 472 GFLOPS FP16 across 128 CUDA cores — between two and three orders of magnitude less capable, sharing 4 GB of memory bandwidth with four A57 cores.

Published Nano benchmarks for YOLOv8n vary widely (6 FPS for naive PyTorch, up to ~30 FPS for a heavily tuned low-resolution TensorRT/DeepStream path). A defensible planning estimate for TensorRT FP16 at 640 px is **12–18 FPS for detection alone**. Layering MiDaS Small on top, in the current strictly-serial loop, yields an estimated **4–6 FPS**.

#### WHY IT BLOCKS

At 5 FPS the system has a 200 ms sampling interval before any inference, fusion or audio latency. A person walking at 1.4 m/s covers ~28 cm between frames. Alerts arrive after the obstacle has been passed, which is worse than useless — it trains the user to distrust the device.

#### RESOLUTION — CUMULATIVE

| # | Measure | Phase | Expected effect |
|---|---|---|---|
| 1 | TensorRT **FP16** for both engines — INT8 is unavailable ( `B20` ) | 2 | 2–3× over PyTorch/ONNX baseline |
| 2 | Reduce detection input 640 → 416 px | 2 | ≈2.3× fewer pixels; costs a few mAP points |
| 3 | Depth at native 256², no upsample ( `B11` ) | 4 | Removes a bicubic upsample of a full-res map per frame |
| 4 | Decouple depth — run MiDaS every 3rd–5th frame | 4 | Removes 60–80 % of depth cost |
| 5 | Threaded stage graph with drop-stale queues | 4 | Overlaps capture / detect / depth / audio |
| 6 | Delete annotation and video writing | 4 | Recovers ARM CPU otherwise spent drawing |
| 7 | Pre-rendered audio vocabulary ( `B14` ) | 5 | Removes TTS synthesis from the CPU entirely |
| 8 | *If required:* replace MiDaS with geometric distance (§6) | — | Removes the depth stage entirely |

#### TARGET

**10–15 FPS end-to-end**, with camera-to-audio latency under 400 ms. Set this as the acceptance criterion, not 29 FPS. If it is not met, escalate the degradation ladder in Section 5.

#### VERIFICATION

`echopath run --profile` reports a per-stage latency histogram over a 10-minute run; end-to-end latency measured physically with the LED-and-microphone method described in Phase 7.

### B7 4 GB shared RAM — TensorRT engine builds will OOM HIGH

#### ROOT CAUSE

The Nano's 4 GB LPDDR4 is shared between CPU and GPU. A stock Ubuntu 18.04 desktop session consumes 1.2–1.5 GB before anything is loaded. TensorRT's builder is memory-hungry: it profiles many tactics per layer, holding several weight copies and intermediate activation buffers resident simultaneously while it searches.

#### WHY IT BLOCKS

`trtexec` will be killed by the OOM reaper mid-build. The failure is abrupt and the message ends up in `dmesg` rather than on stdout, which makes it easy to misread as a TensorRT bug.

#### RESOLUTION

1. **Disable the desktop GUI:** `sudo systemctl set-default multi-user.target`, then reboot. Recovers roughly 0.8–1.2 GB. Use SSH for all work; re-enable with `graphical.target` only if needed for camera debugging.
2. **Create an 8 GB swapfile on the microSD** (not eMMC — see `B2`) and enable `zram`. Slow, but the builder tolerates paging; being killed it does not tolerate.
3. Constrain the builder with `--workspace=512` (MiB) rather than letting it claim everything available.
4. Build the two engines **sequentially, never concurrently**, and reboot between them if the first build leaves memory fragmented.
5. Build engines once and cache the `.plan` files; this cost is not paid at runtime.

#### VERIFICATION

`free -h` shows <600 MB used at idle after GUI removal; both `trtexec` builds complete; `dmesg | grep -i "killed process"` is empty afterwards.

### B8 Alert cooldown and priority are described in the paper but not implemented HIGH

#### ROOT CAUSE

Report Section III-B.3 specifies a minimum 2.5-second wait before a notification may repeat, and priority ordering such that close obstacles always precede medium ones, with only the closest obstacle announced at any time. In `TTS_integrated_model.ipynb` cell 13, `TTSEngine.__init__` accepts `cooldown=2.5` and initialises `self._last_spoken = {}` and `self._last_message = ""` — and **none of those three members is ever read anywhere in the notebook**. There is no `speak()` method at all; the class exposes only `save()`, which writes a WAV for offline muxing.

The offline driver in cell 15 approximates the behaviour with two unrelated mechanisms: `if alert_txt != prev_alert` (suppresses only immediate repeats of an identical string) and a `seen` dictionary that de-duplicates WAV synthesis across the entire video. Neither is time-based, and neither has any meaning for a live stream.

#### WHY IT BLOCKS

The paper's central usability claim — minimal, non-disruptive feedback that avoids cognitive overload, grounded in the auditory-navigation literature it cites — rests on a mechanism that does not exist in the code. Running the current logic live would emit an alert on essentially every processed frame for every persistent obstacle. A user walking down a corridor past a row of columns would hear continuous overlapping speech.

#### RESOLUTION — IMPLEMENT `ALERTS.PY` PROPERLY

- **Temporal hysteresis (entry gate).** An object must be detected in *K of the last M* processed frames (start with K=2, M=3) before it may trigger. Suppresses single-frame false positives, which the reduced input resolution will make more common.
- **Per-key cooldown.** Key on `(label, direction)`, not on the full phrase — otherwise a chair drifting from `medium` to `close` re-announces immediately and defeats the timer. Default 2.5 s, per the paper.
- **Priority pre-emption, not queueing.** If a `close` alert arrives while a `medium` one is playing, cut the medium off. Never build a backlog — a queued alert describes a world that no longer exists.
- **Single alert in flight.** Drop rather than buffer. Maximum queue depth of one, with the newer higher-priority item winning.
- **Class alert policy in config.** `whiteboard`, `monitor`, `notice_board` and `window` are wall-mounted and are never obstacles in a walking path. Detect them (they are useful for the paper's detection metrics) but do not announce them. This directly serves the stated minimal-feedback design goal.
- **Escalation override.** A `close` object *ahead* should bypass cooldown once if its depth is still increasing — that is the genuine collision case.

#### VERIFICATION

Unit tests over synthetic detection sequences: a persistent obstacle produces exactly one alert per 2.5 s window; a close alert pre-empts a playing medium alert; a one-frame spurious detection produces none. Then a live corridor walk with alert timestamps logged and manually reviewed.

### B9 `chair` recall of 0.30 — the most common trip hazard is largely undetected HIGH

**ROOT CAUSE**

Report Table V gives Training v3 `chair` as P = 0.746, R = 0.241, mAP@50 = 0.354. The normalised confusion matrix (Fig. 6) shows recall 0.30 with **0.69 leaking to background** and 0.20 confused with `monitor`. The report acknowledges this as "the biggest limitation" but the system is nonetheless being taken to a wearable prototype.

Note also the regression: v2 achieved `chair` R = 1.000, mAP@50 = 0.995 — but v2 was discarded for overfitting. The v3 augmentation changes (mixup, copy-paste) appear to have disproportionately harmed this class.

**WHY IT BLOCKS**

In an academic building, chairs are the single most frequent floor-level obstacle and the most likely cause of an actual fall. Missing roughly seven in ten of them is a safety-relevant defect, not a metrics footnote. This is the one blocker on the list that concerns physical harm.

**RESOLUTION**

1. **Diagnose before retraining.** Run v2 and v3 side by side on the same held-out chair-heavy footage and inspect failure cases. Determine whether the loss is annotation-driven (chairs under tables, occluded, partially framed) or augmentation-driven (mixup blending chairs into background).
2. **Targeted data collection.** Classroom and corridor footage emphasising chairs: occluded, stacked, side-on, under tables, at varying distances. Re-annotate with a consistent occlusion convention.
3. **Ablate the v3 augmentation.** Retrain v3's configuration with mixup disabled and copy-paste reduced, and compare per-class. If `chair` recovers without the other classes regressing, that is the fix.
4. **Reduce the operating confidence threshold for `chair` specifically.** The pipeline uses a global `CONF_THRESH = 0.40`. Per-class thresholds are trivial to add in `fusion.py`, and for a hazard class, recall should be bought with precision. Report F1-optimal threshold is 0.448 globally; `chair` should sit well below it.
5. **Until fixed, disclose it.** Any user testing must be supervised, and the limitation stated to participants.

**VERIFICATION**

`chair` recall ≥0.75 on a held-out chair-heavy validation set, with no other safety class regressing by more than 0.05 mAP@50.

### B10 Dataset declares 19 classes; the paper reports 17 MEDIUM

**ROOT CAUSE**

`dataset_merged_v3/.../data.yaml` declares `nc: 19`, with `backpack` and `handbag` present in the `names` list at indices 11 and 12. The report states 17 classes throughout (Table II, Table V, the abstract, the conclusion). Both extra classes have zero instances — visible as the two empty rows in the Fig. 6 confusion matrix.

**WHY IT BLOCKS**

Two problems. First, the deployed model has 19 output channels, so **class indices 13–18 in the engine are offset from the paper's 17-class table** — a silent mismapping waiting to happen when `detector.py` maps index to label without the exact same `names` list. Second, it is a straightforward factual inconsistency that a reviewer will find.

**RESOLUTION**

1. Decide the canonical class list — 17, matching the paper.
2. Remove `backpack` and `handbag` from `data.yaml`, set `nc: 17`, and retrain (cheap: 100 epochs at batch 64 on the RTX 4500). Since both classes have zero instances, metrics for the remaining 17 should be essentially unchanged, which also makes this a safe change to make alongside the `B9` chair fix and the Phase 2 imgsz fine-tune — one retrain, three fixes.
3. Emit the class list into `config.yaml` from a single source of truth, and have `detector.py` assert that engine output channel count matches `len(names)` at load time.

#### VERIFICATION

Engine metadata output dimension equals 17 + 4 (box) per anchor; the startup assertion passes; report tables and `config.yaml` agree.

### B11 Full-resolution depth upsampling and per-box percentile recomputation MEDIUM

#### ROOT CAUSE

Three compounding inefficiencies in cell 7, all invisible on an RTX 4500 and all expensive on an A57:

1. `estimate_depth_midas()` runs `torch.nn.functional.interpolate(..., size=frame.shape[:2], mode='bicubic')` — bicubic-upsampling MiDaS's native $256^2$ output to full frame resolution (up to 1920×1080, i.e. ~2.07 M elements).
2. `process_frame()` then normalises that full-resolution map and applies a *second* `cv2.resize` if dimensions disagree.
3. `get_depth_and_zone()` calls `np.percentile(depth_map, 75)` and `np.percentile(depth_map, 50)` **inside the per-detection loop**. These are frame-level constants. With ten detections in frame, the same two full-resolution percentile scans run twenty times per frame.

#### WHY IT BLOCKS

Not correctness — pure throughput. Upsampling discards no information but adds substantial cost, and `np.percentile` performs a partial sort; over 2.07 M elements, twenty times per frame, on an ARM core, this plausibly dominates the entire CPU-side budget.

#### RESOLUTION

- **Keep depth at native 256×256.** Do not upsample. Instead scale bounding-box coordinates *down* into depth space: `x_d = x * (256 / frame_w)`. The inner-50 % crop, the median, and the percentile classification are all scale-invariant operations — the algorithm's semantics are unchanged. This eliminates the bicubic upsample and the redundant resize entirely.
- **Hoist the percentiles to frame level.** Compute the two thresholds once per depth frame and pass them into `get_depth_and_zone()`. Combined with the resolution change, the percentile workload drops from ~2.07 M elements ×2×N detections to 65 k elements ×2×1 — roughly a 600-fold reduction in that operation for a ten-detection frame.
- Guard the crop: at $256^2$ a distant object's box may be only a few pixels, so retain the existing `inner.size == 0` fallback and add a minimum-area check.

#### VERIFICATION

Run both implementations over the 13 test videos on the workstation and diff the emitted alert logs — the proximity/direction decisions must agree on >98 % of frames. Then measure the per-stage delta with `--profile`.

### B12 Proximity zone flicker from per-frame normalisation and adaptive thresholds MEDIUM

#### ROOT CAUSE

The pipeline min–max normalises each depth frame independently (`(raw - raw.min()) / (raw.max() - raw.min())`) and then thresholds against percentiles computed from *that same frame*. Both the value and the yardstick move every frame. When a bright near-field object enters or leaves the frame, the normalisation rescales and every other object's normalised depth shifts, potentially reclassifying static obstacles.

The report's own evidence supports this: Fig. 7 shows MiDaS Small producing **2.3× more unique alerts** than Depth Anything V2 across the same 13 videos — 261 versus 113 — and Section IV-F attributes this directly to adaptive per-frame percentile thresholding. Section IV-G then selects MiDaS anyway on deployment-constraint grounds (25 MB vs 97 MB, offline `torch.hub` availability, simpler integration), which is correct for this hardware — but it means shipping the noisier depth behaviour, so the instability must be handled in software.

#### WHY IT BLOCKS

Zone oscillation produces alert chatter — exactly the cognitive overload the design sets out to avoid — and it interacts badly with the cooldown logic in `B8`, since a flickering object keys differently on each transition.

#### RESOLUTION

- **EMA the thresholds, not the depth map.** Maintain exponentially-weighted running values of the P75 and P50 thresholds with α ≈ 0.15. Cheap, and it stops the yardstick jumping while still adapting to scene changes over ~1 s.
- **Majority vote the emitted zone.** Take the mode of the last three classifications per tracked object before the alert layer sees it.
- **Add threshold hysteresis.** Require crossing P75 + δ to enter `close` but only falling below P75 − δ to leave it (δ ≈ 0.03 normalised). Standard Schmitt-trigger behaviour; eliminates boundary dithering.
- **Optionally, EMA the normalisation bounds too** — smoothing `d_min` / `d_max` across frames removes the rescaling shock when a near object enters frame.
- Log the pre- and post-stabilisation alert counts across the 13 videos. If MiDaS's 261 alerts fall toward Depth Anything V2's 113 without losing genuine close-proximity events, that is a **publishable result** and a direct answer to the report's own open question.

#### VERIFICATION

Unique alert count per video re-measured across all 13 videos; zone-transition rate per tracked object per second drops by >50 %; no genuine close-proximity event present in the baseline is lost.

### B13 MiDaS is loaded via `torch.hub`, which requires internet at startup MEDIUM

#### ROOT CAUSE

Cell 5 calls `torch.hub.load('intel-isl/MiDaS', 'MiDaS_small', trust_repo=True)` and a second `torch.hub.load(..., 'transforms')`. On a cold cache both clone from GitHub and download weights. Report Section IV-G explicitly cites offline-first operation as a hard requirement — one of the four stated reasons MiDaS was chosen over Depth Anything V2 — yet the implementation as written does not satisfy it.

#### WHY IT BLOCKS

On the deployed device this is moot after the port (there is no PyTorch, and the engine is a local `.plan` file), so the deployment risk resolves itself. It matters for two other reasons: the workstation-side export must be reproducible without network access, and the report's offline-first argument is currently contradicted by its own code.

#### RESOLUTION

1. Vendor the MiDaS Small weights and the model definition into the repository under `models/midas_small/`, and load from disk with an explicit `torch.hub.set_dir()` or direct `state_dict` load.
2. Replicate the `small_transform` preprocessing explicitly in NumPy — resize to 256×256, ImageNet mean/std normalisation, HWC→CHW. This must be reimplemented for the device anyway, since there is no `torchvision` there.
3. Pin the exact commit hash of the `intel-isl/MiDaS` repository used, and record it in the report's methodology for reproducibility.

#### VERIFICATION

Export runs to completion on a network-disconnected workstation; NumPy preprocessing output matches the `torch.hub` transform to within 1e-5 on 20 sample frames.

### B14 `pyttsx3` falls back to espeak on ARM — slow, robotic, and CPU-competing MEDIUM

#### ROOT CAUSE

`pyttsx3` is a wrapper, not an engine. On Windows it drives SAPI5; on Linux/ARM it shells out to **espeak** via a subprocess. The voice quality is markedly worse, engine initialisation is slow, and — critically — synthesis runs on the same four A57 cores the detection pipeline needs. The report itself notes a typical phrase takes 1–2 seconds to speak, which is why the producer–consumer thread exists.

#### WHY IT BLOCKS

Not fatal, but it burns scarce CPU on a task that need not be done at runtime at all, and it puts the report's sub-one-second end-to-end latency claim out of reach.

#### RESOLUTION — PRE-RENDER THE CLOSED-SET VOCABULARY

EchoPath's alert grammar is a **closed set**. The phrase template is `"{label} {proximity} {direction}"` and only `close` / `medium` ever produce audio. With 17 classes:

```
17 labels  ×  2 proximity zones (close, medium)  ×  3 directions (left, ahead, right)
=  102 distinct phrases  –  the complete runtime vocabulary
```

Synthesise all 102 once on the workstation, ship them as `assets/audio/*.wav`, and at runtime simply play the matching file through ALSA. Playback latency drops from ~1–2 s of synthesis to well under 50 ms, and runtime TTS CPU cost becomes zero. If the class alert policy from `B8` excludes the four wall-mounted classes, the set shrinks to 78 phrases and roughly 4 MB of 22 kHz mono WAV.

- Use a good voice for the render — **Piper** produces natural neural speech and runs offline on the workstation. Alternatively keep `pyttsx3` /SAPI5 for exact continuity with the recorded results in the report.
- Normalise loudness across all clips so no phrase is quieter than another.
- Trim leading/trailing silence aggressively — every 100 ms of leading silence is 100 ms of alert latency.
- Retain a `Pyttsx3Fallback` for genuinely out-of-vocabulary phrases (diagnostics, "camera disconnected", "battery low").
- Keep the existing producer–consumer thread structure from cell 13 — it is the right shape; only the worker body changes from `engine.say()` to `wav_player.play()`.

#### VERIFICATION

All 102 WAVs render and play; measured dispatch-to-first-sample latency <50 ms; CPU utilisation attributable to audio <2 % during a corridor walk.

### B15 TensorRT engines are not portable across TensorRT version or GPU architecture MEDIUM

**ROOT CAUSE**

A TensorRT `.plan` file is a serialised, hardware-and-version-specific artefact. An engine built by TensorRT 10 on an Ada GPU will not deserialise under TensorRT 8.2 on Maxwell. This also means Ultralytics' convenient `model.export(format='engine')` is unusable here, because it would build on the workstation — and Ultralytics cannot run on the Nano to build it there.

**WHY IT BLOCKS**

It dictates the build topology: **ONNX is the portable artefact, the engine is device-local.** Teams that miss this waste time trying to ship a `.plan` and get an opaque deserialisation error.

**RESOLUTION**

1. Export **ONNX only** on the workstation, with static shapes and batch 1. Use `opset=12` — TensorRT 8.2's ONNX parser becomes unreliable above opset 13.
2. Export with `nms=False` and implement decode + NMS in NumPy in `nms.py`. The `EfficientNMS_TRT` plugin path on JetPack 4.6 adds a dependency on plugin availability for little gain at these detection counts.
3. Build engines on the device with `trtexec`, and commit the exact build command into `tools/` so it is reproducible.
4. `.gitignore` the `engines/` directory. Treat `.plan` files as build output, never as source.
5. Record precision and TensorRT version in the engine filename (e.g. `yolo_416_fp16_trt8201.plan`) and assert both at load time.

**VERIFICATION**

A clean-device rebuild from ONNX reproduces working engines using only the committed commands.

### B16 Thermal throttling and battery budget for wearable operation MEDIUM

**ROOT CAUSE**

Sustained dual-engine inference under `nvpmodel -m 0` (MAXN) with `jetson_clocks` will hold the SoC near its thermal limit. The included fan handles this on a desk; inside a chest-mounted enclosure, airflow is restricted. Separately, the report describes a wearable device but specifies no power source, and 5 V / 4 A is a demanding rail for a portable pack.

**WHY IT BLOCKS**

Throttling silently halves throughput mid-walk, which will be misread as a software regression. And an undersized battery makes the wearable claim untestable.

**RESOLUTION**

- Log `tegrastats` continuously during Phase 7 and plot SoC temperature, GPU clock and instantaneous power against elapsed time. Confirm no clock reduction over a 20-minute run.
- Compare `nvpmodel -m 0` (MAXN, 10 W) against `-m 1` (5 W) with clocks pinned. If 5 W mode still meets the FPS target, take it — it roughly doubles battery life and eliminates the thermal question.
- Ensure the enclosure has through-flow, with the fan intake unobstructed and the heatsink fins aligned to the airflow path.
- **Power budget:** ~10 W sustained. A 20 000 mAh / 74 Wh pack with a 5 V 4 A-capable output (or a USB-PD trigger board set to 5 V/4 A, or 9 V into a buck converter) gives roughly 5–6 hours accounting for conversion losses. Verify the pack sustains 4 A without tripping over-current protection — many nominally 3 A packs will not.
- Add an under-voltage watchdog: L4T reports throttling and rail events via sysfs; announce "power low" through the audio fallback rather than dying silently.

**VERIFICATION**

20-minute continuous walk with no clock throttling logged; measured battery endurance within 20 % of the calculated figure; enclosure surface temperature within safe contact limits.

### B17 The system is notebook-bound with hardcoded absolute paths MEDIUM

**ROOT CAUSE**

The entire pipeline lives in Jupyter cells with literals such as `r"E:\EchoPath\runs\detect\runs\echopath\v3\weights\best.pt"`. There are seven near-duplicate MiDaS pipeline notebooks under `midas_small/` with no indication of which is canonical, and `echopath_test.ipynb` points at three different weight versions across four cells. There is no test suite, no configuration file, and no entry point ( `main.py` is a stub).

**WHY IT BLOCKS**

Nothing here runs unattended on a headless ARM device, and nothing is testable. It also makes the port risky: without a shared code path, the workstation and device implementations will drift, and the Phase 7 fidelity comparison would then measure code divergence rather than quantisation effects.

**RESOLUTION**

Execute Phase 1 in full — the package layout in §2.4, all paths from `config.yaml`, backend abstraction for detector and depth, and unit tests on `fusion.py` and `alerts.py` (both pure functions over NumPy arrays, so entirely testable without a GPU). Freeze and archive the seven notebook variants with a short note recording which experiment each represents.

**VERIFICATION**

`echopath run --source file --input Video_Data/Classroom.MP4` reproduces the notebook's alert log exactly; the test suite passes on both machines; `grep -r "E:\\\\" echopath/` returns nothing.

### B18 Class imbalance of ~1800:1 makes several reported metrics unreliable LOW

**ROOT CAUSE**

Report Fig. 3 gives the Training v2 label distribution: `column` 10 929 instances, `whiteboard` 3 298, `notice_board` 68, `waste_bin` 8, `table` 7. The report states the ratio exceeds 1800:1. Yet Table V reports `table` at mAP@50 = 0.809 and `waste_bin` at 0.828 — figures computed over a handful of validation instances.

**WHY IT BLOCKS**

It does not block deployment. It weakens the paper: per-class metrics over single-digit instance counts have confidence intervals wide enough to be uninformative, and a reviewer will say so.

**RESOLUTION**

- Report per-class validation *instance counts* alongside the metrics in Table V. This is a one-column change and it is honest.
- Mark classes below a threshold (say, <50 validation instances) as statistically underpowered in the caption.
- For deployment, either collect targeted data for the starved classes or drop them from the alert policy — a class the model has seen seven times should not be trusted to warn a blind user.
- Note that `column`'s dominance is genuine to the environment (corridors have pillars), so this is partly a real prior rather than a sampling artefact. Say that explicitly in the paper.

**VERIFICATION**

Table V carries instance counts; the alert policy in `config.yaml` reflects the decision on underpowered classes.

### B19 No failure recovery, no user controls, no service supervision LOW

**ROOT CAUSE**

The notebook assumes an operator watching a screen. A wearable used by a blind person has no operator, no screen, and no keyboard. There is currently no handling for camera disconnection, no restart mechanism, no way for the user to mute or repeat an alert, and no indication that the system is alive.

**WHY IT BLOCKS**

Not a blocker for a lab demonstration; an absolute blocker for the user study the report proposes in Section V-C.

**RESOLUTION**

- `systemd` unit with `Restart=always` and `RestartSec=3`, enabled at boot, running as a non-root user with video and audio group membership.
- An audible **"EchoPath ready"** chime at startup — the only way a blind user can know the system has booted.
- **Physical button on the 40-pin GPIO header:** short press repeats the last alert, long press mutes/unmutes. A blind user cannot reach a keyboard or a screen.
- Camera-disconnect detection with automatic reopen and an audible "camera lost" notice.
- Ring-buffer logging with `journald` size limits so the SD card cannot fill.
- A watchdog on the fusion thread: if no frame is processed for 5 s, announce a fault and restart the pipeline.

**VERIFICATION**

Unplug the camera mid-run — the system announces the fault and recovers when reconnected. Kill the process — `systemd` restarts it within 5 s. Cold boot to first audible alert measured under 60 s.

### B20 The Maxwell GPU has no INT8 support — FP16 is the precision floor MEDIUM

**ROOT CAUSE**

The Jetson Nano's Tegra X1 GPU is Maxwell, **compute capability 5.3**. INT8 inference in TensorRT depends on the DP4A 8-bit dot-product instruction, which NVIDIA introduced with Pascal (compute capability 6.1). Maxwell does not have it, so TensorRT on this device supports **FP32 and FP16 only**. Passing `--int8` to `trtexec` does not error usefully — the builder warns and silently falls back, which is worse than failing, because the resulting engine looks like an INT8 build and performs like an FP16 one.

**WHY IT BLOCKS**

It removes one of the three standard edge-optimisation levers. The usual embedded recipe is *quantise, shrink the input, prune*; here only the middle one is fully available. Concretely it costs an estimated 20–30 % of the throughput that would otherwise have been on the table, which means input resolution and the depth-decoupling in Phase 4 have to carry proportionally more of the load to reach the 10–15 FPS target.

It also deletes work that would otherwise have been scheduled: no calibration set, no entropy calibrator, no calibration cache, and no risk of a calibration/runtime preprocessing mismatch — historically one of the more painful failure modes in edge deployment. On balance the plan gets simpler, not just slower.

**RESOLUTION**

1. Build **both engines in FP16**. Drop the INT8 calibration work from Phase 2 entirely.
2. Recover the lost throughput through input resolution — this makes the Phase 2 resolution ablation more important, not less, since it is now the primary accuracy-versus-latency dial.
3. Verify FP16 is actually being selected: `trtexec` reports the chosen precision per layer with `--verbose`. Confirm no layers silently fell back to FP32.
4. Note that FP16 on Maxwell is genuine 2× throughput over FP32 — the Tegra X1 has full-rate FP16 — so this is still a real and worthwhile speedup, not a token one.
5. If the FP16 target proves unreachable, the next lever is not quantisation but **removing the depth stage** — see §6.

VERIFICATION

`trtexec --fp16 --verbose` shows FP16 tactics selected across the network; measured latency is roughly half the FP32 build; the engine filename records the precision and the loader asserts it.

## 4 · Phase-Wise Execution Plan

Nine phases. Phases 1 and 2 require **no hardware** and should run in parallel with the Phase 0 procurement track, which has external lead time. This parallelism is the single biggest schedule lever available.

| Phase | Title | Hardware needed | Blockers cleared | Gate to proceed |
|---|---|---|---|---|
| **0** | Provisioning & platform baseline | Linux host, board | B2 B3 B4 B5 B16 | BSP in hand, host ready |
| **1** | Notebook → deployable package | None (Windows) | B10 B17 | CLI reproduces notebook output |
| **2** | Model export & optimisation | None (Windows) | B1 B13 B15 | ONNX parity verified |
| **3** | Nano bring-up | Board + host | B1 B2 B3 B4 B7 | Four independent smoke tests pass |
| **4** | Runtime architecture & optimisation | Board | B6 B11 | ≥10 FPS sustained |
| **5** | Audio & alert policy | Board + dongle | B8 B12 B14 | Corridor walk, no chatter |
| **6** | Model accuracy & safety | None (Windows) | B9 B18 | `chair` recall ≥0.75 |
| **7** | On-device validation & measurement | Full system | B6 B16 | Fidelity + latency measured |
| **8** | Productionisation | Full system | B19 | Unattended 1-hour run |

## 0 Provisioning & Platform Baseline

Start immediately · external lead time

**Objective.** Remove every non-software dependency from the critical path. This phase is mostly procurement and correspondence, and it gates Phase 3 — so it must start on day one even though no code is written in it.

### 0.1 PROCUREMENT

- **USB audio dongle** ( `B5` ) — generic USB Audio Class 1.0. Buy two; they are cheap and they fail.
- **IMX219 CSI camera module** with a 15-pin ribbon of adequate length for chest mounting. Confirm 2-lane compatibility with the Eagle 101 connectors.
- **x86_64 machine with Ubuntu 18.04 or 20.04** ( `B3` ) — bare metal or dual boot, not a VM.
- **A2-rated microSD, 128 GB** ( `B2` ) — appears to be on hand; confirm the A2 rating, not merely U3/V30.
- **Battery pack** capable of sustained 5 V / 4 A ( `B16` ), plus a barrel-jack cable of the correct polarity and diameter.
- Micro-USB or USB-C to USB Type-B cable for the recovery port.

### 0.2 VENDOR ENGAGEMENT ( `B4` )

- Scan both QR codes on the retail box: *How to Assemble* and *How to Flash JetPack (BSP)*.
- Email `sales@tannatechbiz.com` requesting: the BSP archive for JetPack 4.6.x, the exact `flash.sh` board-configuration name, the documented rootfs-on-microSD procedure for this carrier, and the CSI camera compatibility list.
- Record the unit serial (1010425035310) in the correspondence.

### 0.3 DECISIONS TO LOCK BEFORE FLASHING

| Decision | Recommendation | Rationale |
|---|---|---|
| Rootfs location | **microSD** | Eliminates `B2` outright; boot-speed cost only |

| Decision | Recommendation | Rationale |
|---|---|---|
| Desktop environment | **Headless** ( `multi-user.target` ) | Recovers 0.8–1.2 GB of 4 GB ( `B7` ) |
| Swap | **8 GB file on microSD + zram** | Required for TensorRT engine build ( `B7` ) |
| Camera interface | **CSI /** `nvarguscamerasrc` | Hardware ISP, zero-copy; USB UVC burns CPU on MJPEG decode |
| Power mode | Start **MAXN**, evaluate 5 W in Phase 7 | Measure before committing ( `B16` ) |
| JetPack components | Full, minus samples/docs/VisionWorks | Saves ~2 GB even with the SD rootfs |

**EXIT CRITERIA**

BSP archive and board-config name received from the vendor · Ubuntu 18.04/20.04 x86 host provisioned and SDK Manager installed · all BOM items in hand · rootfs/swap/camera decisions recorded in `docs/platform-decisions.md` .

# 1 Notebook → Deployable Package

Windows workstation · no hardware

**Objective.** Convert the notebook pipeline into a tested, configurable Python package with a backend abstraction, while preserving the fusion algorithm's semantics exactly. Everything in this phase runs on the existing Windows machine and can start today.

### 1.1 BUILD THE PACKAGE SKELETON

Create the layout from §2.4. Move all constants into `config.yaml`:

```
model:
  yolo_weights:  runs/detect/runs/echopath/v3/weights/best.pt   # dev backend
  yolo_engine:   engines/yolo_416_fp16_trt8201.plan             # device backend (B20: no INT8)
  depth_engine:  engines/midas_small_256_fp16_trt8201.plan
  imgsz:         416
  depth_size:    256
detection:
  conf_thresh:   0.40
  conf_overrides:                # per-class, see B9
    chair:       0.25
  iou_thresh:    0.45
depth:
  close_percentile:   75
  medium_percentile:  50
  threshold_ema_alpha: 0.15      # B12
  hysteresis_delta:    0.03      # B12
  inner_crop_ratio:    0.50      # preserve cell 7 semantics
geometric:                       # §6 — fallback / comparison arm
  enabled:        false
  f_px:           null           # calibrate with the deployment camera
  camera_height:  1.35           # metres, chest mount
  real_heights:   {person: 1.70, door: 2.00, chair: 0.90, table: 0.75}
runtime:
  depth_every_n_frames: 3        # B6
  detect_every_n_frames: 1
alerts:
  cooldown_sec:      2.5         # B8 — paper Section III-B.3
  hysteresis_k:      2
  hysteresis_m:      3
  zone_vote_window:  3           # B12
  silent_classes: [whiteboard, monitor, notice_board, window]   # B8
audio:
  vocabulary_dir: assets/audio
  alsa_device:    default
```

### 1.2 LIFT THE FUSION CORE VERBATIM

Move `clamp_bbox`, `get_depth_and_zone` and `generate_alerts` from cell 7 into `fusion.py` **with their logic unchanged** at this stage. The `B11` optimisation and the `B12` stabilisation come later, as deliberate, separately-measured changes — not as incidental edits during a refactor. This ordering is what lets you attribute any behavioural difference to a specific change.

### 1.3 DEFINE THE BACKEND INTERFACES

```
class Detector(ABC):
    @abstractmethod
    def infer(self, frame_bgr) -> list[Detection]: ...
    @property
    @abstractmethod
    def names(self) -> list[str]: ...

class DepthEstimator(ABC):
    @abstractmethod
    def infer(self, frame_bgr) -> np.ndarray:   # normalised [0,1], native resolution
        ...
```

Implement `UltralyticsDetector` and `MidasTorch` now. The TensorRT implementations land in Phase 3, behind the same interfaces, and nothing above them changes.

### 1.4 FIX THE CLASS-COUNT INCONSISTENCY ( `B10` )

Set `nc: 17` in `data.yaml`, remove `backpack` and `handbag`. Defer the retrain itself into Phase 6 so it can be batched with the `B9` chair fix and the Phase 2 resolution fine-tune — one training run, three fixes.

### 1.5 TEST SUITE

- Unit tests on `fusion.py`: synthetic depth maps and boxes; assert zone and direction. Include boxes overrunning frame edges, degenerate one-pixel boxes, and empty inner crops.
- Unit tests on `alerts.py` (once Phase 5 writes it) using a mock clock.
- **Golden-output regression test:** run the CLI over `Video_Data/Classroom.MP4` and assert the alert log matches the notebook's recorded output byte for byte. This is the anchor for everything that follows.

### 1.6 HOUSEKEEPING

- Archive the seven `midas_small/*.ipynb` variants under `experiments/` with a README recording what each one tested.
- Add `.gitignore` entries for `engines/`, `runs/`, `*.zip`, `Video_Data/`. Note the repository currently has a 3.9 GB `Video_Data.zip` and a 121 MB `Depth Analysis V2.zip` untracked — decide explicitly whether these belong in Git LFS or outside the repo entirely.
- Split `pyproject.toml` into `[project.optional-dependencies]` groups: `dev` (torch, ultralytics, transformers) and `device` (numpy, pycuda) so the device install is unambiguous.

**EXIT CRITERIA**

`echopath run --source file --input Video_Data/Classroom.MP4` reproduces the notebook alert log exactly · all unit tests pass · `grep -r "E:\\\\" echopath/` returns nothing · configuration fully externalised.

# 2 Model Export & Optimisation

Windows workstation · no hardware

**Objective.** Produce verified ONNX artefacts and the empirical data needed to choose the deployment operating point. Also runs entirely on the workstation, in parallel with Phase 0.

### 2.1 EXPORT YOLOV8N TO ONNX

```
model.export(
    format='onnx',
    imgsz=416,        # chosen in 2.3
    opset=12,         # TensorRT 8.2's parser degrades above 13
    simplify=True,
    dynamic=False,    # static shapes — B15
    batch=1,
    nms=False,        # decode + NMS in numpy — B15
)
```

Export at every candidate resolution so the ablation can be run against real engines later.

### 2.2 EXPORT MIDAS SMALL TO ONNX ( `B13` )

- Static 1×3×256×256 input — MiDaS Small's native transform size.
- Load weights from the vendored local copy, not `torch.hub`.
- Reimplement `small_transform` in NumPy and assert agreement with the torch transform to within 1e-5 across 20 frames.
- **Parity check:** cosine similarity between PyTorch and ONNX Runtime depth maps over 20 frames sampled from `Video_Data` must exceed 0.99. Fail the build if not — a silently wrong depth map is the hardest defect to diagnose later.

### 2.3 RESOLUTION ABLATION — NEW PAPER CONTENT

Run `model.val(imgsz=N)` on the v3 weights for N ∈ {640, 512, 416, 320}. No retraining is needed for a first pass; a short fine-tune at the chosen resolution recovers most of the loss and can be folded into the Phase 6 retrain.

| imgsz | Relative pixels | mAP@50 | Nano latency (ms) | Decision |
|---|---|---|---|---|
| 640 | 1.00× | 0.869 (published) | measure in Phase 3 | baseline |
| 512 | 0.64× | — | — | — |
| 416 | 0.42× | — | — | expected choice |
| 320 | 0.25× | — | — | fallback |

This table, once populated with measured Nano latencies from Phase 3, becomes **Table VIII** of the report and is the deployment-constraint analysis Section III-C currently promises but answers with desktop numbers.

### 2.4 PRECISION STRATEGY ( `B20` )

**FP16 for both engines.** INT8 is not available on Maxwell — see `B20`. This removes the calibration set, the entropy calibrator and the whole class of calibration/runtime preprocessing mismatches, so Phase 2 is simpler than originally scoped; the cost is that input resolution becomes the primary latency dial rather than a secondary one.

Verify with `trtexec --fp16 --verbose` that FP16 tactics are actually selected throughout and no layers quietly fell back to FP32.

### 2.5 PROTOTYPE THE GEOMETRIC DEPTH BASELINE (§6)

Build `tools/proto_geometric.py` and run it over the 13 videos. This is workstation-only work with no hardware dependency, it produces the approach-tracking plots described in §6, and it gives you a fully-formed fallback in hand

*before* Phase 4 tells you whether you need one. It also stands on its own as a third comparison arm against MiDaS and Depth Anything V2 for the paper.

### 2.6 PRE-RENDER THE ALERT VOCABULARY ( `B14` )

Write `tools/render_vocabulary.py` to synthesise all 102 phrases (or 78 after the class alert policy) to `assets/audio/`, loudness-normalised and silence-trimmed. Naming convention `{label}_{proximity}_{direction}.wav` so runtime lookup is a string format with no table.

**EXIT CRITERIA**

ONNX files for YOLO at all four resolutions and MiDaS at $256^2$ · parity checks pass · mAP-vs-imgsz table populated (latency column pending Phase 3) · geometric baseline prototyped and plotted · 102 WAVs rendered and audibly checked.

## 3 Nano Bring-Up

Board + Linux host · gated on Phase 0

**Objective.** Get a working, correctly-configured device and prove each subsystem *independently*. Do not port the pipeline until all four smoke tests pass — debugging a full pipeline on an unproven platform conflates software bugs with device-tree problems and is the most common way this kind of project stalls.

### 3.1 FLASH ( `B2` , `B3` , `B4` )

1. Assemble per the vendor's *How to Assemble* guide; seat the module and connect the fan header.
2. Enter recovery mode (typically jumper `FC REC` to `GND` , then power on). Confirm with `lsusb | grep -i nvidia` → an `APX` device.
3. Flash using the **vendor BSP** with rootfs targeted at the microSD. *Use the board-config name supplied by Tanna TechBiz.*
4. Install JetPack components over the network with `sdkmanager` or `apt` , omitting samples, documentation and VisionWorks.

### 3.2 CONFIGURE ( `B7` )

```
sudo systemctl set-default multi-user.target     # headless — recovers ~1 GB
sudo fallocate -l 8G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile   # persist in /etc/fstab
sudo nvpmodel -m 0                               # MAXN
sudo jetson_clocks                               # pin clocks
sudo systemctl disable bluetooth snapd           # if unused
```

### 3.3 THE FOUR INDEPENDENT SMOKE TESTS

| # | Subsystem | Test |
|---|---|---|
| 1 | TensorRT — detection | `trtexec --onnx=yolov8n_416.onnx --fp16 --workspace=512 --saveEngine=engines/yolo_416_fp16.plan` builds and reports mean latency. **Record this number** — it populates the Phase 2 ablation table. |
| 2 | TensorRT — depth | `trtexec --onnx=midas_small_256.onnx --fp16 --workspace=512 --saveEngine=engines/midas_256_fp16.plan` likewise. |
| 3 | Camera | `gst-launch-1.0 nvarguscamerasrc num-buffers=300 ! 'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! nvvidconv ! fakesink -v` sustains 30 FPS with no dropped-buffer warnings. |
| 4 | Audio ( `B5` ) | `aplay -l` lists the USB device; a rendered vocabulary WAV plays audibly through earphones; the device resolves correctly after three reboots. |

### 3.4 DEVICE RUNTIME ENVIRONMENT ( `B1` )

```
python3 -m venv --system-site-packages ~/echopath-env    # inherit tensorrt + cv2
source ~/echopath-env/bin/activate
pip install pycuda                                        # numpy stays at system 1.19
python3 -c "import tensorrt, pycuda.autoinit, cv2, numpy; print(tensorrt.__version__)"
```

Verify `cv2.getBuildInformation()` reports `GStreamer: YES` . **Never** `pip install opencv-python` — it shadows JetPack's build and silently removes GStreamer and CUDA support, which breaks the CSI camera path.

### 3.5 IMPLEMENT THE TENSORRT BACKENDS

`TensorRTDetector` and `MidasTensorRT` behind the Phase 1 interfaces. Points of care:

- Allocate **pinned host memory** for bindings — pageable transfers cost measurably on this platform.
- Use **FP16 I/O bindings** where the engine allows, to avoid host-side casts.

- Give each engine its **own CUDA stream**, so Phase 4 can overlap them.
- Assert at load time that the engine's TensorRT version and output dimensions match expectations ( `B10` , `B15` ).
- Implement decode + NMS in NumPy in `nms.py` and unit-test it on the workstation against Ultralytics' output for identical inputs.

**EXIT CRITERIA**

Board boots headless from microSD with >40 GB free and 8 GB swap active · all four smoke tests pass · both engines built and cached · `echopath run --source file` executes end-to-end on the device at *any* frame rate · measured latencies recorded in the ablation table.

## 4 Runtime Architecture & Optimisation

On device · the core engineering

**Objective.** Take the working-but-slow device pipeline from ~5 FPS to the 10–15 FPS target. Apply the four changes below **one at a time, measuring after each** — the point is to know which change bought what, both for engineering judgement and because these measurements are paper content.

### 4.1 STOP UPSAMPLING THE DEPTH MAP (`B11`)

The highest-yield single change. Keep depth at native 256×256; scale bounding boxes down into depth space rather than scaling depth up into frame space. Hoist the two `np.percentile` calls out of the per-detection loop to frame level.

```
# Before  (cell 7) — per detection, on a full-resolution map
close_thresh  = np.percentile(depth_map, 75)      # ~2.07M elements, ×N detections
medium_thresh = np.percentile(depth_map, 50)

# After — once per depth frame, on the native 256x256 map
close_t, medium_t = np.percentile(depth_256, [75, 50])    # 65k elements, ×1
sx, sy = 256 / frame_w, 256 / frame_h                      # scale boxes into depth space
```

The inner-50 % crop, the median, and the percentile comparison are all scale-invariant, so the algorithm's semantics are preserved. Validate that claim empirically: alert-log agreement over the 13 videos must exceed 98 %.

### 4.2 DECOUPLE DEPTH FROM DETECTION (`B6`)

Depth changes far more slowly than object identity. Run YOLO every frame and MiDaS every 3rd–5th, reusing the last depth map in between. The current `PROCESS_EVERY_N = 2` gates *both* together, which is the wrong granularity — it halves detection responsiveness to save depth cost.

Sweep `depth_every_n_frames` ∈ {1,2,3,5,8} and measure both FPS and alert-log agreement against `n = 1`. There will be a knee; take the value just before agreement degrades.

### 4.3 THREADED STAGE GRAPH

```
[CSI capture] → q1 → [YOLO TRT] ─┐
                                 ├→ q3 → [fuse + alert policy] → q4 → [audio]
      every Nth frame → [MiDaS TRT] ─┘

All queues: maxsize=1, drop-oldest on full.
```

- **Depth-1, drop-stale queues throughout.** Never buffer. For a walking user, a 600 ms-old alert describes a world that no longer exists — dropping it is strictly better than delivering it late.
- Capture, detection, depth, fusion and audio as five stages. Detection and depth run on separate CUDA streams so they can overlap.
- Bear in mind Python's GIL: the CUDA calls release it during device execution, which is where the overlap comes from. If the NumPy post-processing proves to be a GIL bottleneck, move NMS into a `multiprocessing` worker — measure before assuming.

### 4.4 DELETE THE ANNOTATION PATH

Remove `cv2.rectangle`, `cv2.putText` and `cv2.VideoWriter` from the production path entirely — they are pure overhead when the user is blind. Retain behind `--debug-view`, default off. Keep it working, though: it is essential for demonstrations and for the paper's figures.

### 4.5 INSTRUMENT

Build `profiler.py` as a ring buffer of per-stage timings, reporting p50/p95/p99 per stage plus end-to-end. Expose via `--profile`. Without this, optimisation is guesswork; with it, the paper gets a per-stage latency breakdown table that no comparable published system provides.

**EXIT CRITERIA**

**≥10 FPS sustained** over a 10-minute run with both engines active · per-stage p95 latencies recorded · alert-log agreement with the workstation >95 % · a table attributing the speedup to each of the four changes.

## 5 Audio & Alert Policy

On device · highest value-to-effort ratio

**Objective.** Make the system's actual user-facing behaviour match what the paper describes. This is the phase that turns a detector into an assistive device, and it is mostly ordinary software engineering with no hardware risk.

### 5.1 SHIP THE PRE-RENDERED VOCABULARY ( `B14` )

Replace runtime synthesis with WAV playback. Keep the producer–consumer thread from cell 13 — the structure is right; only the worker body changes. Retain `Pyttsx3Fallback` for diagnostics and out-of-vocabulary phrases.

### 5.2 WRITE `ALERTS.PY` ( `B8` )

Implement, in this order, testing each against synthetic sequences with a mock clock:

1. **Entry hysteresis** — K-of-M frame presence before an object may trigger.
2. **Per-`(label, direction)` cooldown** at 2.5 s. Keying on the full phrase instead would let a `medium` → `close` transition bypass the timer.
3. **Priority pre-emption** — `close` cuts off a playing `medium`.
4. **Single alert in flight** — drop, never queue; newest high-priority item wins.
5. **Class alert policy** — `silent_classes` from config are detected but never announced.
6. **Escalation override** — a `close` object *ahead* with increasing depth may bypass cooldown once.

### 5.3 STABILISE PROXIMITY CLASSIFICATION ( `B12` )

EMA on the P75/P50 thresholds ($\alpha \approx 0.15$), three-frame majority vote on the emitted zone, and Schmitt-trigger hysteresis of $\delta \approx 0.03$ on the zone boundaries.

Then re-measure unique alert counts across all 13 videos and compare against the report's Fig. 7 baseline of 261 for MiDaS versus 113 for Depth Anything V2. If stabilisation closes that gap without losing genuine close-proximity events, it is a direct, publishable answer to the open question the report leaves in Section IV-G.

### 5.4 FIELD-TUNE WITH A SIGHTED WALKER FIRST

Before any blindfolded or visually-impaired participant testing, have a sighted team member wear the device through the Academic Block with alert timestamps logged, and annotate each alert as useful / redundant / missed / wrong. This is a cheap, fast loop and it will surface policy problems no unit test can.

**EXIT CRITERIA**

All 102 phrases play with <50 ms dispatch latency · alert policy unit tests pass · a full corridor walk produces no overlapping speech and no repeat within 2.5 s · unique alert counts re-measured across the 13 videos · sighted-walker annotation shows >80 % of alerts rated useful.

## 6 Model Accuracy & Safety

Windows workstation · parallelisable

**Objective.** Fix the safety-relevant detection gaps before anyone wears the device. Batch all model changes into a single retrain.

### 6.1 THE CHAIR PROBLEM ( `B9` ) — THE PRIORITY ITEM

1. **Diagnose first.** v2 achieved `chair` R = 1.000; v3 achieves 0.241. Run both on identical chair-heavy footage and inspect failures. The v3 augmentation changes (mixup, copy-paste) are the prime suspect.
2. **Ablate the augmentation.** Retrain v3's configuration with mixup disabled. If `chair` recovers without regressing other classes, that is the fix and it costs one training run.
3. **Collect targeted data** if ablation is insufficient: chairs occluded, stacked, side-on, under tables, at range. Consistent occlusion annotation convention.

4. **Per-class confidence threshold.** Add `conf_overrides` support in `fusion.py` and drop `chair` below the global 0.40. For a hazard class, buy recall with precision.

### 6.2 BATCH INTO ONE RETRAIN

Combine into a single training run: the 17-class fix ( `B10` ), the augmentation ablation ( `B9` ), and a fine-tune at the Phase 2 target resolution. Then re-export ONNX and rebuild engines on the device. Re-run the golden-output regression test afterwards.

### 6.3 METRIC HONESTY ( `B18` )

Add per-class validation *instance counts* to Table V and mark classes below ~50 instances as statistically underpowered. Note explicitly that `column` 's dominance reflects a genuine environmental prior — corridors have pillars — rather than purely a sampling artefact.

**EXIT CRITERIA**

`chair` recall ≥0.75 on a held-out chair-heavy set · no safety class regressed by >0.05 mAP@50 · 17-class model exported and rebuilt on device · Table V carries instance counts.

## 7 On-Device Validation & Measurement

Full system · produces the paper's new results

**Objective.** Produce defensible measurements of the deployed system. Everything in this phase is publishable.

### 7.1 DEPLOYMENT FIDELITY — THE HEADLINE NEW METRIC

Run all 13 Academic Block videos through the Nano pipeline in file-input mode, and diff the resulting alert logs against the workstation output using `tools/compare_alert_logs.py`. Report an **agreement rate**, broken down by cause:

- Alerts present on both, identical zone and direction → agreement.
- Present on both, different zone → quantisation or frame-skip effect.
- Present on workstation only → missed detection (FP16 rounding or resolution loss).
- Present on Nano only → false positive introduced by reduced precision or resolution.

This quantifies exactly what FP16, 416 px and frame-skipping cost in *user-visible* terms rather than in mAP, which is a more meaningful statement for an assistive system and is not something comparable published work reports.

### 7.2 PERFORMANCE MEASUREMENT

- Sustained FPS over 20 minutes, with the distribution — not just the mean.
- Per-stage latency breakdown, p50/p95/p99, from `profiler.py`.
- **True end-to-end latency, measured physically.** Place an LED in the camera's field of view and a microphone at the earphone. Trigger the LED, record both, and measure the gap in the audio file. Do not estimate this by summing stage timings — that omits buffering, scheduling and ALSA latency, which is precisely where a sub-one-second claim is won or lost.
- `tegrastats` logged throughout: SoC temperature, GPU/CPU clocks, instantaneous power, RAM. Confirm no throttling (`B16`).
- Peak RSS and whether swap is touched during steady-state operation.

### 7.3 WEARABLE VALIDATION

- Battery endurance under continuous operation, against the ~10 W calculation.
- Enclosure surface temperature; confirm safe against-body contact.
- Chest-mount camera geometry: field of view, tilt angle, and whether the floor immediately ahead — where trip hazards are — is actually in frame. This is easy to get wrong and it determines whether the system can see the chairs at all.
- Compare MAXN against 5 W mode; if 5 W meets the FPS target, take it.

### 7.4 THEN, AND ONLY THEN, THE USER STUDY

The report's Section V-C proposes a structured user study with visually impaired participants against a white-cane baseline. That study is only meaningful once `B9` (chair recall) is fixed and Phases 5, 8 are complete. Running it earlier will measure the prototype's defects rather than the concept's merit.

**EXIT CRITERIA**

Fidelity agreement rate measured and broken down by cause · physically-measured end-to-end latency · 20-minute thermal and power log with no throttling · battery endurance figure · all Jetson numbers ready for the paper.

## 8 Productionisation

Full system · required before user testing

**Objective.** Make the device usable by someone who cannot see a screen and has no operator (`B19`).

- `systemd` **service** — `Restart=always`, `RestartSec=3`, enabled at boot, running as a non-root user in the `video` and `audio` groups.
- **Audible readiness chime** at startup. A blind user has no other way to know the system has booted.

- **GPIO button on the 40-pin header** — short press repeats the last alert, long press toggles mute. Debounce in software; announce the mute state change audibly.
- **Camera-disconnect recovery** — detect, announce "camera lost", retry with backoff, announce recovery.
- **Fusion-thread watchdog** — if no frame is processed for 5 s, announce a fault and restart the pipeline.
- **Bounded logging** — `journald` with a size cap plus a rotating alert log, so the SD card cannot fill.
- **Power monitoring** — read L4T's throttling and rail sysfs entries; announce "power low" rather than dying silently.
- **Documentation** — a one-page operator guide (power on, what the chime means, what the button does, what to do if it goes quiet) and a rebuild runbook covering flash → engine build → service install.

**EXIT CRITERIA**

Cold boot to first audible alert <60 s · unattended 1-hour walk with no manual intervention · camera unplug/replug recovers automatically · process kill recovers within 5 s · operator guide and rebuild runbook written.

## 5 · Degradation Ladder & Risk Register

### 5.1 If Phase 4 does not reach 10 FPS

Escalate in order. Each rung costs something; take the cheapest rung that meets the target.

| Rung | Action | Cost | Expected gain |
|---|---|---|---|
| 1 | Detection at 320 px FP16; depth every 5th frame | A few mAP points, coarser depth timing | ~1.7× over 416/every-3 |
| 2 | Reduce capture resolution to 640×480 at source | Narrower effective FOV or more downscaling | Lowers ISP and memcpy cost |
| 3 | Depth only on frames that contain a detection | Loses scene-level percentile context on empty frames | Large in sparse corridors |
| 4 | **Drop MiDaS entirely — use projective geometry instead. Fully specified in §6.** | Loses monocular depth; degrades for unknown-size and wall-mounted objects | Removes the entire depth stage — likely 2×+ |
| 5 | Move to a **Jetson Orin Nano** | Hardware cost | ~80× AI performance; JetPack 6 / Python 3.10 means the existing PyTorch code runs nearly as-is, collapsing Phases 2–4 |

> **On rung 4**
> Dropping MiDaS is less drastic than it sounds, and it may not even be a downgrade. It is important enough — and self-contained enough to build and evaluate before the board is flashed — that it gets its own section. **See §6.**

> **On rung 5 — the honest recommendation**
> Essentially every difficulty in this document traces back to the Nano being EOL on Python 3.6 with a Maxwell GPU. On an Orin Nano, the existing `torch 2.4` + Ultralytics code runs almost unchanged, `model.export(format='engine')` works directly, there is no 16 GB storage crisis, and Phases 2 and 3 shrink to a day's work. If the budget permits it and the deadline is tight, this is the rational choice. If the Eagle 101 is what you have, the plan above is sound — it is simply more work.

### 5.2 Risk register

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Vendor BSP unavailable or undocumented | Medium | High | Contact vendor in Phase 0 day one; escalate through the reseller; NVIDIA developer forums as backup. Do not start flashing without it. |
| Board bricked by an incorrect flash | Low | High | Never guess the board config. Recovery mode is generally re-enterable; document the exact procedure before the first attempt. |
| Reduced input resolution degrades safety classes disproportionately | Medium | High | Phase 7 fidelity comparison is designed to detect exactly this. Step back to 512 px for detection if `chair` / `stairs` / `wet_floor_sign` regress. Note INT8 is not a variable here ( `B20` ). |
| 10 FPS unreachable even after Phase 4 | Medium | Medium | Degradation ladder §5.1. |
| CSI camera incompatible with the carrier | Low | Medium | Confirm the compatibility list with the vendor in Phase 0. USB UVC is a workable, slower fallback. |

| Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|
| Thermal throttling inside the enclosure | Medium | Medium | Phase 7 measures it explicitly. 5 W mode and enclosure redesign are the levers. |
| Chair recall not recoverable by augmentation ablation | Medium | High | Falls back to targeted data collection — schedule the collection session in parallel rather than waiting for the ablation result. |
| microSD wear or corruption | Low | Medium | Bounded logging, `noatime`, and a documented rebuild runbook so a reflash is a one-hour recovery rather than a project setback. |

## 6 · Alternative Depth Strategy — Geometric Distance Estimation

This is rung 4 of the degradation ladder, specified in full. It replaces MiDaS with projective geometry: instead of asking a neural network how far away something is, compute it from the object's apparent size and its known real-world size. It is cheap enough to be effectively free, it can be built and evaluated on the Windows workstation before the board is even flashed, and — for the reason set out in §6.4 — it may be an *upgrade* rather than a fallback.

### 6.1 The principle

For a pinhole camera, apparent size and distance are related by a single expression:

```
Z  =  (f_px × H_real) / h_px

Z       distance to the object                                    (metres)
f_px    focal length in pixels  =  (image_height / 2) / tan(VFOV / 2)
H_real  the object's known real-world height                      (metres)
h_px    bounding-box height                                       (pixels)
```

One division per detection. No model, no weights, no GPU, no depth stage. The output is an **absolute distance in metres** — a stronger statement than MiDaS produces, since MiDaS yields a scene-relative inverse-depth ranking with no metric scale.

#### WORKED APPROACH — IMX219 AT 720P (VFOV ≈ 48.8°, F_PX ≈ 794), CHAIR (H_REAL = 0.90 M)

| Distance | Box height | Growth per frame @ 1.4 m/s, 13 FPS | Error per px of box noise |
|---|---|---|---|
| 4.0 m | 179 px | +5 px | ±2.2 cm |
| 3.0 m | 238 px | +9 px | ±1.3 cm |
| 2.0 m | 357 px | +21 px | ±0.6 cm |
| 1.5 m | 476 px | +36 px | ±0.3 cm |
| 1.0 m | 715 px | +80 px | ±0.14 cm |

Two properties matter. First, walking 4 m → 1 m takes about 2.1 s, which at 13 FPS gives roughly **28 distance updates — one every 10 cm of travel**. Second, because `dZ/dh = −f·H/h²`, **precision improves quadratically as the object gets closer**: halving the distance quarters the error. That is exactly the right characteristic for a hazard warning system — approximate when the obstacle is far and irrelevant, sharp when it is close and dangerous.

### 6.2 Applicable classes

Most of the deployment class set has tightly constrained real-world height:

| Class | H_real | Class | H_real | Usable cue |
|---|---|---|---|---|
| person | 1.70 m | water_dispenser | 1.30 m | height + ground |
| door | 2.00 m | waste_bin | 0.70 m | height + ground |
| chair | 0.90 m | wet_floor_sign | 0.60 m | height + ground |
| table | 0.75 m | potted_plant | 0.80 m | height + ground |
| fire_extinguisher | 0.60 m | elevator | 2.00 m | height only (wall-mounted) |
| column | — | stairs | — | **ground-plane only** — always truncated |
| whiteboard, monitor, notice_board, window | | | | **neither cue works** |

> **The convenient coincidence**
> The four classes where geometry fails outright — `whiteboard`, `monitor`, `notice_board`, `window` — are wall-mounted, are never obstacles in a walking path, and are **exactly the four classes already placed in `silent_classes`** by the alert policy in `B8`. They are detected but never announced, so the inability to range them costs nothing. The overlap between "geometry fails here" and "we do not care here" is close to total.

### 6.3 The second cue — ground-plane contact

The height cue fails on truncated boxes. A person at 1.5 m projects to 900 px in a 720 px frame; the box is clipped and `h_px` becomes meaningless precisely when the hazard is most urgent. Columns are truncated always. A second, complementary estimator is therefore required rather than optional:

```
Z_ground  =  (camera_height × f_px) / (y_bottom − y_horizon)

camera_height   mounting height of the chest-worn camera    (metres, measured once)
y_bottom        bottom edge of the bounding box             (pixels)
y_horizon       image row of the horizon, from camera pitch (pixels, calibrated once)
```

Given the camera's height and pitch, the image row at which an object meets the floor maps directly to its distance along the ground. This is standard monocular ADAS practice, and it is *better* than the height cue for anything floor-standing — including the columns and stairs the height cue cannot handle at all.

**Fusion rule.** Use ground-plane when the bottom edge is visible and the class is floor-standing; use height when the object is fully framed; when both are available and disagree, **take the closer estimate**. For an assistive device, biasing toward "nearer than it really is" is the correct failure direction.

The existing `clamp_bbox()` already knows when a box was clipped — that flag is precisely the signal for switching cues, so the plumbing is largely in place.

### 6.4 Why this tracks approach *better* than the current pipeline

This is the substantive argument, and it is not about speed.

Consider what `get_depth_and_zone()` actually does: it classifies an object by comparing its median depth against `np.percentile(depth_map, 75)` — a threshold computed from **the same frame the object appears in**. As the user approaches an obstacle, that obstacle occupies a progressively larger fraction of the image. Once it exceeds roughly a quarter of the pixels, **the object's own depth values begin to determine the P75 threshold it is being measured against.**

The object raises the bar it has to clear. In a sparse corridor where a single chair is the dominant near-field object, its classification can stall at `medium`, or oscillate between `medium` and `close`, during the final approach — the moment at which a correct answer matters most. The classification is relative to the scene, and by then the scene is largely just the chair.

Geometric distance has no such feedback loop. Two metres is two metres whether the corridor is empty or crowded, and the estimate falls monotonically as the box grows. The EMA and hysteresis fixes in `B12` mitigate a symptom; geometry removes the mechanism that produces it.

### 6.5 What must be added — frame-to-frame identity

The pipeline currently performs no tracking: each frame yields an independent detection list with no notion of "the same chair". To state that a distance is *decreasing*, detections must be associated across frames. A greedy IoU or centroid matcher suffices — a few dozen lines, negligible cost — and it unlocks three capabilities the current system cannot express at all:

- **Smoothing.** An EMA or small Kalman filter over `Z` removes per-frame box jitter, turning a noisy sequence into a clean approach curve.
- **Closing speed.** `dZ/dt` — distinguishing an obstacle being approached from one being passed.
- **Time-to-contact.** `TTC = Z / (dZ/dt)`. For a walking user this is arguably more actionable than raw distance: "*chair, two seconds*" conveys more than "*chair, medium*". It is also what makes the escalation override specified in `B8` implementable at all, since that rule requires knowing depth is still decreasing — which requires identity across frames.

### 6.6 Limitations

| Limitation | Detail and mitigation |
|---|---|
| **Detection dropout** | You cannot range what you do not detect. At `chair` recall 0.30 ( `B9` ) the track shreds and no estimator survives that. The geometry is sound; the detector is the binding constraint. This is the third independent place in this document where `B9` emerges as the critical path. |
| **Intra-class height variance** | `chair` spans a low stool to a tall lab chair; `table` varies similarly. Expect ±30 % error. The proximity zones are coarse enough to absorb it, but it argues for continuing to announce zones rather than reporting metres to the user. |
| **Camera pitch oscillation** | A chest-mounted camera bobs with each step, perturbing `y_horizon` and therefore the ground-plane cue; the height cue is largely immune. Temporal smoothing handles most of it. If it proves severe, an IMU over I²C on the 40-pin header allows direct compensation — but measure before adding that scope. |
| **Requires calibration** | `f_px` , `camera_height` and `y_horizon` must be measured with the **deployment camera**. One clip of a door or a person at a tape-measured distance gives `f_px = Z · h_px / H_real` . |
| **No range for unmodelled objects** | Anything outside the class list, or any class without a height prior, gets no estimate. MiDaS is class-agnostic; geometry is not. For a fixed 17-class assistive system this is an acceptable trade. |

### 6.7 Validation — do this now, on the workstation

The whole approach can be evaluated before the Jetson is flashed, using the existing 13 videos and the already-loaded `yolo_model` in `TTS_integrated_model.ipynb`. No hardware dependency, roughly a day of work.

#### DECOUPLE SHAPE FROM SCALE

The existing videos were shot on a phone, so `f_px` is unknown — and it is also *irrelevant to deployment*, since the real calibration must use the IMX219. But `f_px` is a single multiplicative constant: it sets the scale, not the shape. The question being tested — *does the estimate fall smoothly and monotonically during an approach?* — is answerable without it. Plot `H_real / h_px` in relative units, confirm the curve behaves, and pin the constant later with the deployment camera.

#### PROCEDURE

1. Run YOLO over a clip containing a clear approach; for each detection compute `h_px` , look up `H_real` , and flag boxes touching a frame edge as truncated.
2. Associate detections across frames by greedy IoU matching (threshold ≈ 0.3, same class only).
3. Plot relative distance against frame index for the longest-lived tracks, alongside raw box height.
4. Report the **truncation percentage** across all tracks.

#### READING THE OUTPUT

- **Curve shape.** A genuine approach should be a smooth downward hyperbola, steepening as it closes. Smooth and monotonic means the concept works. A sawtooth means detection jitter, which §6.5's smoothing addresses.

- **Truncation percentage.** This decides how much weight the ground-plane cue must carry. Below ~15 %, the height cue carries the system. Above ~40 % — likely, given `column` dominates the dataset at 10 929 instances — ground-plane becomes the primary estimator, not the backup.
- **Track lifetimes.** Short, fragmented tracks are detection dropout. Filter to `chair` specifically and `B9` becomes visible directly, as broken tracks.

#### THE COMPARISON FIGURE

Once the shape looks right, run the same clip through the existing `process_frame()` and overlay MiDaS's per-object median depth on the same frame axis. That single plot — geometric distance falling smoothly while the percentile-relative value does whatever it does as the object comes to dominate the frame — settles whether §6 is a fallback or an upgrade, and it is a publishable figure either way.

> **Promotion criteria — when this stops being a fallback**
> Adopt geometric estimation as the **primary** depth strategy if, on the 13-video corpus: (1) approach curves are monotonic on >90 % of tracks lasting more than 15 frames; (2) unique alert counts fall from MiDaS's 261 toward Depth Anything V2's 113 without losing any genuine close-proximity event; and (3) the truncation rate is manageable once the ground-plane cue is added. If all three hold, the depth stage is deleted outright — blockers `B11`, `B12` and `B13` resolve simultaneously, roughly 12 ms per frame returns to the budget, and the system no longer has to load, convert or justify a second neural network.

---

## 7 · Required Amendments to the Report

Deployment work will invalidate or strengthen several claims. These changes should be made regardless of how far the deployment proceeds — most are corrections to statements that are already inaccurate.

| Location | Issue | Required change |
|---|---|---|
| **Table I** | 29 FPS / 20.6 ms / 6 MB are RTX 4500 Ada measurements, but the table is used to argue Jetson deployment feasibility. This is the most exposed claim in the paper. | Relabel explicitly as workstation measurements, and add a parallel **Jetson Nano table** with Phase 7 numbers. A reviewer will find this; preempt it. |
| **Abstract, §III-C** | "Real-time throughput (>25 FPS)" stated as a satisfied deployment constraint. | Restate against measured on-device throughput, or scope the claim to the workstation and state the on-device figure separately. |
| **§III-B.3** | Describes 2.5 s cooldown and priority ordering that are not implemented ( `B8` ). | Either implement (Phase 5) and keep the text, or mark as designed-not-yet-implemented. Phase 5 implements it, so keep the text and cite the measured behaviour. |
| **§III-A** | "Delivered to the user with less than one second of delay" — never measured. | Replace with the physically-measured Phase 7 figure. |
| **Table II / §III-D** | States 17 classes; `data.yaml` declares 19 ( `B10` ). | Reconcile to 17, retrain, and state the class list once from a single source. |
| **Table V** | Per-class metrics reported over as few as 7 validation instances ( `B18` ). | Add an instance-count column; flag underpowered classes in the caption. |
| **§IV-G** | Argues MiDaS was chosen partly for offline operation, but the implementation uses `torch.hub`, which needs the network ( `B13` ). | Vendor the weights and amend the text to describe what the code does. |

| Location | Issue | Required change |
|---|---|---|
| **§V-A** | `chair` at 0.30 recall is described as a limitation while the system advances to a wearable ( `B9` ). | Report the Phase 6 improvement, or state plainly that the prototype is not suitable for unsupervised use until it is fixed. |
| **New content generated by this work — all of it strengthens the paper** | | |
| *New Table VIII* | — | mAP@50 vs input resolution vs measured Jetson latency (Phases 2, 3). |
| *New Table IX* | — | Per-stage latency breakdown on Jetson Nano (Phase 4). Rarely reported in comparable work. |
| *New Table X* | — | **Deployment fidelity**: workstation-vs-edge alert agreement, broken down by cause (Phase 7). A genuinely novel metric for assistive systems. |
| *New §* | — | Alert stabilisation results — unique alert counts before and after EMA/hysteresis, against the Fig. 7 baseline (Phase 5). This directly answers the open question §IV-G leaves about MiDaS's 2.3× alert rate. |
| *New §* | — | Power, thermal and battery endurance for the wearable (Phase 7). Section V-C currently lists deployment as future work; this converts it into a result. |
| *New § / Fig.* | — | **Geometric distance estimation as a third depth arm** (§6), evaluated against MiDaS Small and Depth Anything V2 on the same 13 videos. Adds a zero-cost, metrically-scaled baseline to a comparison that currently contains only two learned models, and supplies the approach-tracking figure that neither learned model can produce. |

> **The strategic point**
>
> This deployment is not merely engineering overhead — it is the missing experimental section. The paper's central claim is that EchoPath is suitable for low-cost edge deployment, and it currently supports that claim with desktop GPU measurements and a comparison against RT-DETR. Real Jetson Nano numbers, a deployment-fidelity analysis, and a per-stage latency breakdown turn the weakest part of the paper into its most distinctive contribution.

## Appendix A · Command Reference

### A.1 WORKSTATION — EXPORT, ABLATION AND GEOMETRIC BASELINE

```
# YOLOv8n -> ONNX at each candidate resolution
python tools/export_onnx.py --weights runs/detect/runs/echopath/v3/weights/best.pt \
       --imgsz 640 512 416 320 --opset 12 --simplify --static --no-nms

# MiDaS Small -> ONNX (256x256, local weights, parity-checked)
python tools/export_onnx.py --midas --size 256 --weights models/midas_small/ --check-parity

# Resolution ablation
python tools/ablate_imgsz.py --weights .../best.pt --sizes 640 512 416 320 --data .../data.yaml

# Geometric depth baseline — approach-tracking plots over all 13 videos (§6)
python tools/proto_geometric.py --videos Video_Data/ --plot --out out/geometric/

# Pre-render the alert vocabulary
python tools/render_vocabulary.py --classes 17 --out assets/audio/ --normalise --trim
```

### A.2 DEVICE — CONFIGURATION

```
sudo systemctl set-default multi-user.target        # headless (B7)
sudo fallocate -l 8G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile      # add to /etc/fstab
sudo nvpmodel -m 0 && sudo jetson_clocks            # MAXN + pinned clocks
free -h && df -h /                                  # verify B2 and B7
tegrastats --interval 1000 --logfile ~/tegrastats.log
```

### A.3 DEVICE — ENGINE BUILDS

```
/usr/src/tensorrt/bin/trtexec \
    --onnx=onnx/yolov8n_416.onnx \
    --fp16 --verbose \
    --workspace=512 --avgRuns=100 \
    --saveEngine=engines/yolo_416_fp16_trt8201.plan

/usr/src/tensorrt/bin/trtexec \
    --onnx=onnx/midas_small_256.onnx \
    --fp16 --workspace=512 --avgRuns=100 \
    --saveEngine=engines/midas_256_fp16_trt8201.plan
```

FP16 only — INT8 is unavailable on Maxwell ( `B20` ). Build sequentially, never concurrently ( `B7` ). Record the reported mean latency from each — it populates the Phase 2 ablation table. Check the `--verbose` log to confirm no layers fell back to FP32.

### A.4 DEVICE — SMOKE TESTS

```
# Camera (B4)
gst-launch-1.0 nvarguscamerasrc num-buffers=300 ! \
  'video/x-raw(memory:NVMM),width=1280,height=720,framerate=30/1' ! \
  nvvidconv ! fakesink -v

# Audio (B5)
aplay -l && aplay -D default assets/audio/person_close_ahead.wav

# Runtime stack (B1)
python3 -c "import tensorrt, pycuda.autoinit, cv2, numpy as np; \
            print(tensorrt.__version__, cv2.__version__, np.__version__)"
python3 -c "import cv2; print('GStreamer' in cv2.getBuildInformation())"
```

### A.5 DEVICE — RUN AND PROFILE

```
echopath run --source csi --profile                          # production path
echopath run --source file --input Video_Data/Classroom.MP4 \
             --log-alerts out/nano_classroom.jsonl           # Phase 7 fidelity input
python tools/compare_alert_logs.py out/nano_*.jsonl out/workstation_*.jsonl
```

---

## Appendix B · Alert Vocabulary Enumeration

The runtime alert grammar is a closed set, which is what makes pre-rendering viable ( `B14` ).

```
Template:    "{label} {proximity} {direction}"

labels      = 17 deployment classes
proximity   = {close, medium}          # 'far' never produces audio (paper §III-B.1)
direction   = {left, ahead, right}     # vertical-thirds centroid rule

Full set    : 17 × 2 × 3 = 102 phrases
After silent_classes (whiteboard, monitor, notice_board, window) : 13 × 2 × 3 = 78

Storage     : ~78 × 50 KB ≈ 4 MB at 22.05 kHz mono PCM
Naming      : assets/audio/{label}_{proximity}_{direction}.wav
Lookup      : pure string format — no lookup table, no synthesis, no allocation
```

Additional non-grammar clips required by Phases 5 and 8: `system_ready` , `camera_lost` , `camera_restored` , `muted` , `unmuted` , `power_low` , `fault_restarting` .

---

## Appendix C · Suggested Sequencing

Durations are indicative and assume part-time work by a small team. The critical insight is the parallel track: Phases 1, 2 and 6 need no hardware and should proceed while Phase 0 procurement is in flight.

| Week | Hardware track | Software track (Windows, no hardware) |
|---|---|---|
| 1 | **Phase 0** — contact vendor, order dongle/camera/battery, provision Ubuntu host | **Phase 1** — package skeleton, config, fusion lift, tests |
| 2 | Phase 0 continues — await BSP and parts | **Phase 1** complete · **Phase 2** begins — ONNX exports, parity checks |
| 3 | **Phase 3** — flash, configure, four smoke tests | **Phase 2** — ablation, geometric baseline (§6), vocabulary render |
| 4 | **Phase 3** — TensorRT backends, first end-to-end run on device | **Phase 6** — chair diagnosis and augmentation ablation |
| 5 | **Phase 4** — depth-at-256, decoupling, threading, measure each | **Phase 6** — batched retrain, re-export |
| 6 | **Phase 4** complete · **Phase 5** — audio and alert policy | Rebuild engines from the retrained model |
| 7 | **Phase 5** — stabilisation, sighted-walker tuning | Draft the new paper sections |
| 8 | **Phase 7** — fidelity, latency, thermal, battery | Populate Tables VIII–X |
| 9 | **Phase 8** — systemd, GPIO button, recovery, runbook | Report amendments (§7) |
| 10+ | User study (gated on Phase 6 chair fix) | Final paper revision |

## Appendix D · Sources

- Tanna TechBiz — Eagle 101 Nano Base Full System product page and retail packaging (unit serial 1010425035310).
- Tanna TechBiz — Eagle 101 Carrier Board specification (2× MIPI CSI-2, 4× USB 3.0, HDMI/DP, 16 GB eMMC, 5 V/4 A).
- NVIDIA JetPack 4.6.x / L4T R32.7 release documentation — Python 3.6, CUDA 10.2, TensorRT 8.2.
- NVIDIA Jetson PyTorch wheel index — `torch-1.10.0a0+git36449ea-cp36-cp36m-linux_aarch64.whl` as the terminal cp36 build.
- Ultralytics installation requirements — Python ≥3.8.
- Published YOLOv8n Jetson Nano TensorRT benchmarks (range 6–30 FPS depending on precision, resolution and runtime).
- *EchoPath: Real-Time Assistive Navigation System for Visually Impaired Using Computer Vision* — Jadhav, Salot, Chauhan, Bansal, Mishra, Saharan. Gati Shakti Vishwavidyalaya. Tables I–VII, Figs. 1–8.
- EchoPath repository: `TTS_integrated_model.ipynb` (cells 3, 5, 7, 13, 15), `Yolo_train.py`, `pyproject.toml`, `dataset_merged_v3/data.yaml`, `runs/detect/runs/echopath/v{1,2,3}/`.

---

EchoPath — Jetson Nano (Eagle 101) Deployment Plan · Revision 1.0 · Prepared 10 August 2026 · 20 blockers logged (3 critical, 6 high, 9 medium, 2 low) across 9 phases. Estimates for on-device throughput are planning figures derived from published benchmarks for comparable Maxwell-class hardware and must be replaced with measured values from Phases 3, 4 and 7.